Dawa Saathi

Medicine Awareness Telegram Bot — Technical Reference (Tables)

A reference of the system in clean tables: the request flow, the tech stack, the database schema, Redis usage, the services, environment variables, and the safety layers. Pair this with DOCUMENTATION.md for the full narrative.

1. What happens when a user uploads a photo

Step	Stage	What happens
1	Receive	Telegram delivers the photo as a file_id (a reference, not the bytes).
2	Download	We pick the largest image size and download the bytes (with retry).
3	Rate limit	Check Redis: has the user used their 3 free scans today? If yes, stop.
4	Vision read	Image is base64-encoded and sent to Claude vision with the extraction prompt.
5	Confidence gate	If not a medicine OR confidence < 0.75: ask for a clearer photo and refund the scan.
6	Ask purpose	Save the reading in Redis; ask the user WHY they take it (tappable buttons).
7	User answers	User taps a button or types the reason; buttons are removed to stop double-taps.
8	Cache lookup	Check Redis then Postgres for medicine + purpose. If found, return instantly.
9	openFDA	Optionally verify the ingredient against the public FDA database (resilient).
10	Awareness	Claude writes a simple summary, led by the user's stated purpose.
11	Store	Save the answer to Postgres (permanent) and Redis (30-day cache).
12	Reply	Format as a Telegram message, append the disclaimer, send (with retry).

2. Technology stack

Component	Technology	Why
Language	TypeScript on Node.js 20	Type safety, large ecosystem
Bot transport	Telegram Bot API (long polling)	No public URL needed; works in Docker
Vision (read label)	Claude Sonnet	Accurate reading — safety-critical step
Awareness (write text)	Claude Haiku	Cheaper; medicine already confirmed
Web framework	Express	Lightweight; only used for health checks
Database	PostgreSQL 16	Durable storage of medicines + usage

Component	Technology	Why
Cache / rate limit	Redis 7	Fast, in-memory, auto-expiring keys
Drug verification	openFDA (public API)	Cross-check ingredients; optional
Logging	Pino	Structured JSON logs
Config validation	Zod	Fails fast if any env var is wrong
Containerization	Docker + Docker Compose	One command runs app + DB + Redis

3. Database schema (PostgreSQL)

Table: medicines — the permanent cache of analyzed medicines

Column	Type	Purpose
id	SERIAL (PK)	Auto-incrementing unique ID
ingredient_key	VARCHAR(64) UNIQUE	Fingerprint of ingredients + purpose; the lookup key
medicine_name	VARCHAR(255)	The medicine name
analysis	JSONB	The full AI answer stored as JSON
hit_count	INTEGER	How many times this medicine was looked up
created_at	TIMESTAMPTZ	When first stored
updated_at	TIMESTAMPTZ	When last updated

Table: usage_log — daily scan count per user

Column	Type	Purpose
id	SERIAL (PK)	Auto ID
telegram_user_id	BIGINT	Which user
scan_date	DATE	Which day (YYYY-MM-DD)
scan_count	INTEGER	Scans done that day
(user_id, scan_date)	UNIQUE	One row per user per day

Table: scan_log — analytics for every scan attempt

Column	Type	Purpose
id	SERIAL (PK)	Auto ID
telegram_user_id	BIGINT	Which user
outcome	VARCHAR(32)	extraction_ok, low_confidence, etc.
cache_hit	BOOLEAN	Was it served from cache?
read_confidence	REAL	Vision confidence score (0-1)
duration_ms	INTEGER	How long the scan took
created_at	TIMESTAMPTZ	When

Indexes (for fast lookups)

Index	On	Speeds up
idx_medicines_name	LOWER(medicine_name)	Searching medicines by name
idx_usage_user_date	(telegram_user_id, scan_date)	Today's scan count for a user
idx_scan_log_created	created_at DESC	Sorting analytics by time

4. Redis usage (three jobs)

Job	Key pattern	Expiry	What it does
Medicine cache	med:v1:<key>	30 days	Stores generated answers so repeat scans skip the AI call
Rate limit	rl:daily:v1:<user>:<date>	Midnight	Atomic counter; blocks after 3 scans/day; auto-resets at midnight
Pending extraction	pending:v1:<chatId>	10 minutes	Holds the reading while waiting for the user's purpose answer

5. The services (each does one job)

Service	Responsibility
telegram.service.ts	Connect to Telegram; download files; send messages (all with retry)
vision.service.ts	Stage 1 — base64 the image, send to Claude, return reading + confidence
awareness.service.ts	Stage 2 — send confirmed medicine + purpose to Claude, return summary
fda.service.ts	Verify the active ingredient against openFDA (resilient, optional)
cache.service.ts	Two-tier cache (Redis to Postgres) + pending-extraction storage
ratelimit.service.ts	Daily quota: reserve, refund, record, report usage
medicine.service.ts	Orchestrator — runs both stages and ties all services together

6. Key environment variables

Variable	Default	Meaning
TELEGRAM_BOT_TOKEN	(required)	From @BotFather
ANTHROPIC_API_KEY	(required)	From console.anthropic.com
EXTRACTION_MODEL	claude-sonnet-4-6	Model used to read the label
CLAUDE_MODEL	claude-haiku-4-5	Model used to write awareness text
EXTRACTION_CONFIDENCE_THRESHOLD	0.75	Below this, refuse and ask for clearer photo
FREE_DAILY_SCAN_LIMIT	3	Free scans per user per day
MEDICINE_CACHE_TTL_SECONDS	2592000	Cache lifetime (30 days)
ENABLE_FDA_LOOKUP	true	Turn openFDA verification on/off

7. The four safety layers

Layer	What it protects against
1. Split pipeline	Reading and interpreting are separate; the AI never guesses-and-explains in one step
2. Extraction prompt	Forbids guessing; warns about look-alike drug names; demands honest confidence
3. Confidence gate	Below 0.75, refuse to answer and refund the scan — no confident wrong answers
4. Awareness prompt + disclaimer	Never diagnoses/prescribes/doses; disclaimer added in code on every message

Guiding principle: a confident wrong answer is more dangerous than no answer. Every layer above serves that principle.